Demand over capacity

Congestion is a ratio: demand divided by capacity

The idea

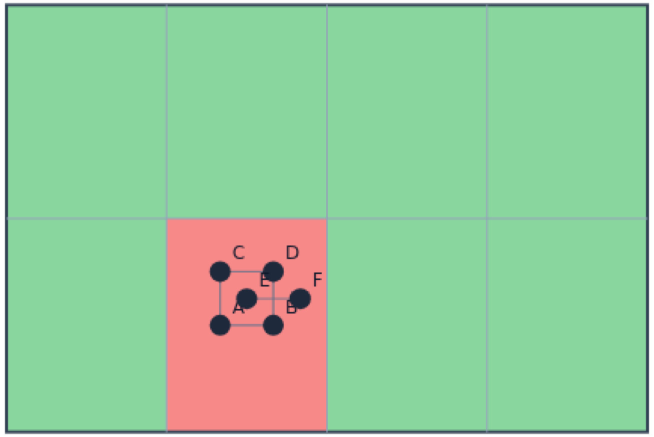
- Cong[i][j] equals demand[i][j] over Cap
- Values above one are oversubscribed
- The hottest GCell is the argmax
- On the congested seed, expect the center columns to light up first

Demand / Cap

CONGESTION MAP

Demand / Cap

Step 1 / 5 · ratio · module02-05-congestion-map



Explanation

Congestion is a ratio per GCell—values above one are oversubscribed.

- Heat = ratio

```
Cap=2
```

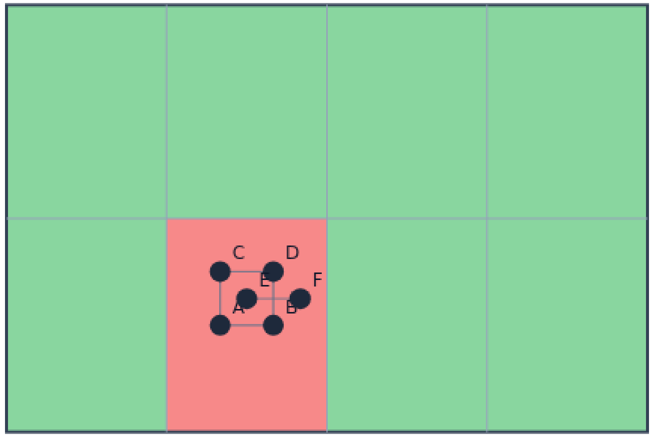
Demand / Cap

Hottest tile

CONGESTION MAP

Hottest tile

Step 2 / 5 · hot · module02-05-congestion-map



Explanation

Argmax over the matrix with fixed scan order for stable goldens.

- Name (i,j)

```
Center on seed
```

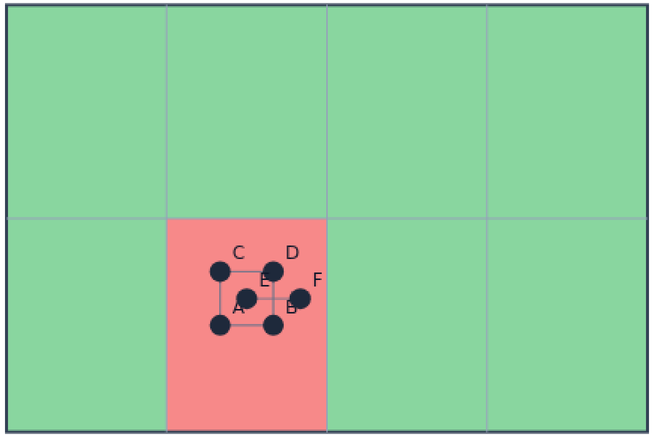
Hottest tile

Read the colors

CONGESTION MAP

Read the colors

Step 3 / 5 · legend · module02-05-congestion-map



Explanation

Hotter colors mean higher congestion; use metrics for exact floats.

- Don’t eyeball only

```
Regression uses numbers
```

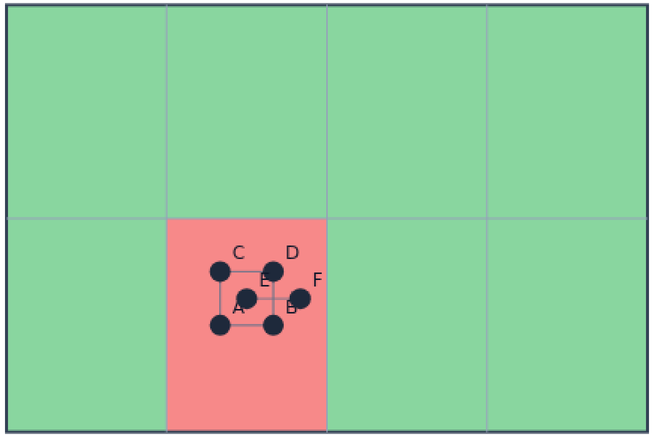
Read the colors

Move the hotspot

CONGESTION MAP

Move the hotspot

Step 4 / 5 · move · module02-05-congestion-map



Explanation

Dragging cells can move which tile is hottest.

- Learner state

```
Challenges score positions
```

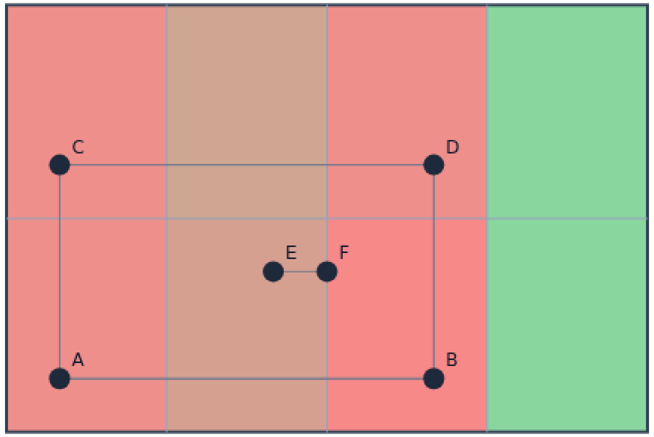
Move the hotspot

Cooler map

CONGESTION MAP

Cooler map

Step 5 / 5 · spread · module02-05-congestion-map



Explanation

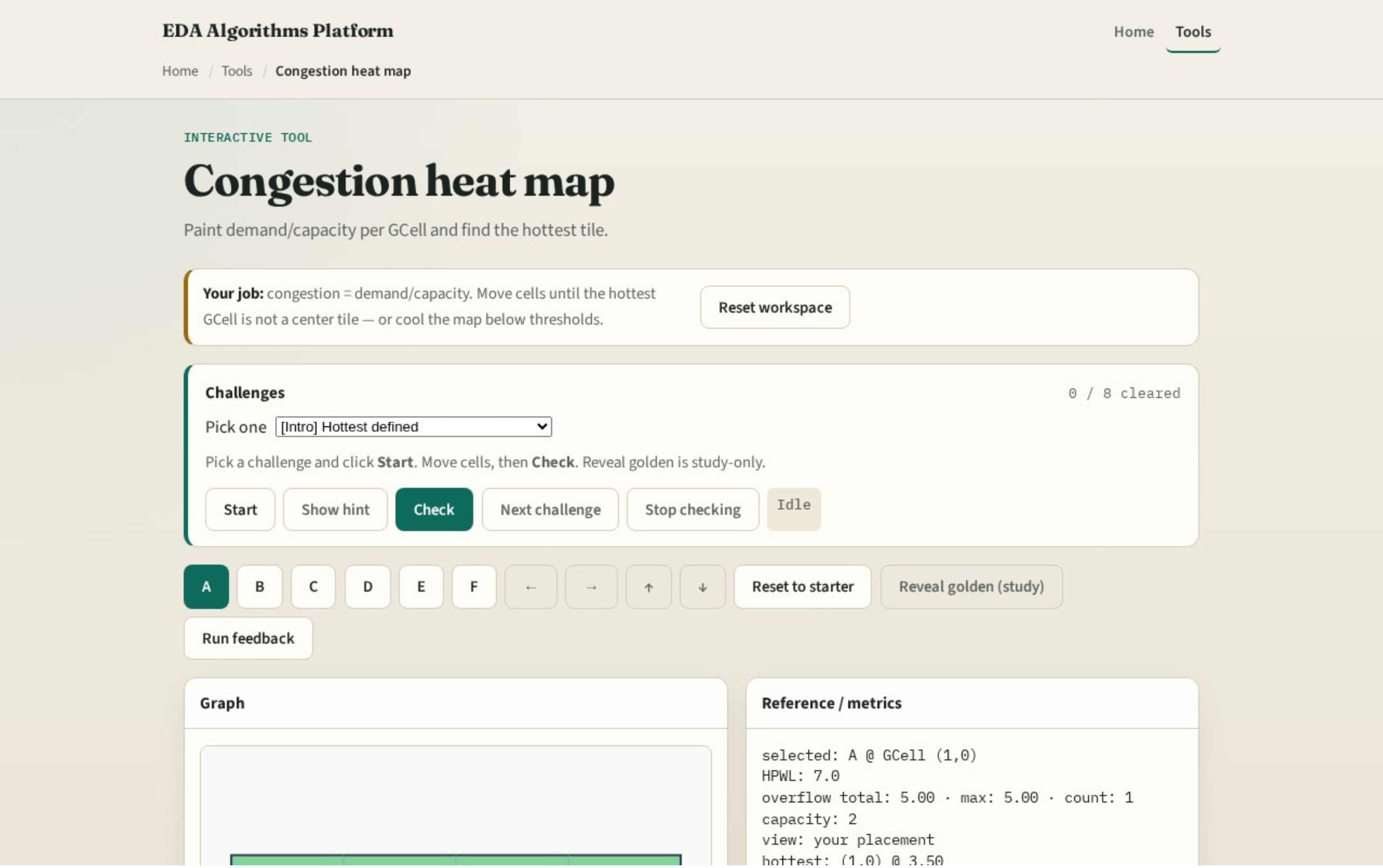
Spread placement lowers peak ratios.

- Toward routing

```
Overflow lab next
```

Cooler map

Browser lab track



Browser lab starter

Implement track

- Build `congestion_map(demand, capacity)` returning the ratio matrix and hottest index
- Print hottest for both placement seeds

Pitfalls

- Dividing by zero capacity
- Pick a fixed scan order

Your turn

- Finish the lab
- Next: overflow metrics compress the map into totals you can regress